SIMATS ENGINEERING

CO2-ASSESSMENT TOOL 1

BY:

192571052_JEROLDSAMUEL J

1.Given a set of relations and sample data, write SQL queries to retrieve specific information using joins andsubqueries, and explain the logic used. Bloom's Level: BL3 – Apply

```
mysql> SELECT * FROM Student;
+-----------+-------------+------------+
| StudentID | StudentName | Department |
+-----------+-------------+------------+
|       101 | John        | CSE        |
|       102 | Alice       | ECE        |
|       103 | Bob         | CSE        |
|       104 | David       | EEE        |
+-----------+-------------+------------+
4 rows in set (0.02 sec)

mysql> SELECT * FROM Marks;
+-----------+---------+-------+
| StudentID | Subject | Marks |
+-----------+---------+-------+
|       101 | DBMS    |    85 |
|       102 | DBMS    |    78 |
|       103 | DBMS    |    92 |
|       104 | DBMS    |    67 |
+-----------+---------+-------+
4 rows in set (0.00 sec)

mysql> SELECT s.StudentID,
    ->        s.StudentName,
    ->        s.Department,
    ->        m.Subject,
    ->        m.Marks
    -> FROM Student s
    -> JOIN Marks m
    -> ON s.StudentID = m.StudentID;
+-----------+-------------+------------+---------+-------+
| StudentID | StudentName | Department | Subject | Marks |
+-----------+-------------+------------+---------+-------+
|       101 | John        | CSE        | DBMS    |    85 |
|       102 | Alice       | ECE        | DBMS    |    78 |
|       103 | Bob         | CSE        | DBMS    |    92 |
|       104 | David       | EEE        | DBMS    |    67 |
+-----------+-------------+------------+---------+-------+
4 rows in set (0.12 sec)
```

```
mysql> SELECT StudentName
    -> FROM Student
    -> WHERE StudentID IN
    -> (
    ->     SELECT StudentID
    ->     FROM Marks
    ->     WHERE Marks > 80
    -> );
+-------------+
| StudentName |
+-------------+
| John        |
| Bob         |
+-------------+
2 rows in set (0.01 sec)

mysql> SELECT s.StudentName,
    ->        m.Marks
    -> FROM Student s
    -> JOIN Marks m
    -> ON s.StudentID = m.StudentID
    -> WHERE m.Marks =
    -> (
    ->     SELECT MAX(Marks)
    ->     FROM Marks
    -> );
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| Bob         |    92 |
+-------------+-------+
1 row in set (0.05 sec)
```

```
mysql> SELECT s.StudentName,
    ->        s.Department,
    ->        m.Marks
    -> FROM Student s
    -> JOIN Marks m
    -> ON s.StudentID = m.StudentID
    -> WHERE s.Department='CSE';
+-------------+------------+-------+
| StudentName | Department | Marks |
+-------------+------------+-------+
| John        | CSE        |    85 |
| Bob         | CSE        |    92 |
+-------------+------------+-------+
2 rows in set (0.00 sec)

mysql> SELECT s.StudentName,
    ->        m.Marks
    -> FROM Student s
    -> JOIN Marks m
    -> ON s.StudentID = m.StudentID
    -> WHERE m.Marks >
    -> (
    ->     SELECT AVG(Marks)
    ->     FROM Marks
    -> );
+-------------+-------+
| StudentName | Marks |
+-------------+-------+
| John        |    85 |
| Bob         |    92 |
+-------------+-------+
2 rows in set (0.05 sec)
```

2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse

Relational Algebra Expression

Retrieve the names of CSE students who scored more than 80 marks.

π StudentName, Marks

(

σ Department = 'CSE' ∧ Marks > 80

(

Student ⋈ Student.StudentID = Marks.StudentID Marks

)

)

Explanation

- **⋈ (Join):** Combines the Student and Marks tables using StudentID.
- **σ (Selection):** Filters records where Department = 'CSE' and Marks > 80.
- **π (Projection):** Displays only the StudentName and Marks columns.

Output

StudentName	Marks
John	85
Bob	92

Operators Used

Operator	Meaning
σ	Selection (Filter rows)
π	Projection (Select columns)
⋈	Join (Combine relations)
∧	AND condition

This is a complete **BL4 – Analyse** answer demonstrating how relational algebra retrieves records that satisfy **multiple conditions across two related relations**.

3.Given a real-world scenario (e.g., banking or studentsystem), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create

```
mysql> SELECT * FROM Student;
+-----------+-------------+------------+------+
| StudentID | StudentName | Department | Age  |
+-----------+-------------+------------+------+
|       101 | John        | CSE        |   20 |
|       102 | Alice       | ECE        |   21 |
|       103 | Bob         | CSE        |   22 |
+-----------+-------------+------------+------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Course;
+----------+------------+-----------+-------+
| CourseID | CourseName | StudentID | Marks |
+----------+------------+-----------+-------+
|        1 | DBMS       |       101 |    85 |
|        2 | Java       |       102 |    78 |
|        3 | DBMS       |       103 |    92 |
+----------+------------+-----------+-------+
3 rows in set (0.05 sec)

mysql> SELECT s.StudentName,
    ->        s.Department,
    ->        c.CourseName,
    ->        c.Marks
    -> FROM Student s
    -> JOIN Course c
    -> ON s.StudentID = c.StudentID;
+-------------+------------+------------+-------+
| StudentName | Department | CourseName | Marks |
+-------------+------------+------------+-------+
| John        | CSE        | DBMS       |    85 |
| Alice       | ECE        | Java       |    78 |
| Bob         | CSE        | DBMS       |    92 |
+-------------+------------+------------+-------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT StudentName, Department
    -> FROM Student
    -> WHERE Department='CSE';
+-------------+------------+
| StudentName | Department |
+-------------+------------+
| John        | CSE        |
| Bob         | CSE        |
+-------------+------------+
2 rows in set (0.00 sec)

mysql> UPDATE Course
    -> SET Marks = 90
    -> WHERE StudentID = 101;
Query OK, 1 row affected (0.05 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Course;
+----------+------------+-----------+-------+
| CourseID | CourseName | StudentID | Marks |
+----------+------------+-----------+-------+
|        1 | DBMS       |       101 |    90 |
|        2 | Java       |       102 |    78 |
|        3 | DBMS       |       103 |    92 |
+----------+------------+-----------+-------+
```

```
mysql> SELECT * FROM Student;
+-----------+-------------+------------+------+
| StudentID | StudentName | Department | Age  |
+-----------+-------------+------------+------+
|       101 | John        | CSE        |   20 |
|       102 | Alice       | ECE        |   21 |
|       103 | Bob         | CSE        |   22 |
+-----------+-------------+------------+------+
3 rows in set (0.00 sec)
```

4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze

```
mysql> SELECT * FROM Employee;
+-------+---------+-------------+--------+
| EmpID | EmpName | Department  | Salary |
+-------+---------+-------------+--------+
|   101 | Rahul   | HR          |  30000 |
|   102 | Priya   | IT          |  45000 |
|   103 | Arun    | IT          |  50000 |
|   104 | Kiran   | Finance     |  35000 |
|   105 | Meena   | IT          |  55000 |
+-------+---------+-------------+--------+
5 rows in set (0.00 sec)

mysql> SELECT AVG(Salary)
    -> FROM Employee
    -> WHERE Department='IT';
+-------------+
| AVG(Salary) |
+-------------+
|  50000.0000 |
+-------------+
1 row in set (0.00 sec)

mysql> SELECT EmpName, Salary
    -> FROM Employee
    -> WHERE Salary >
    -> (
    ->     SELECT AVG(Salary)
    ->     FROM Employee
    ->     WHERE Department='IT'
    -> );
+---------+--------+
| EmpName | Salary |
+---------+--------+
| Meena   |  55000 |
+---------+--------+
1 row in set (0.00 sec)
```

5. Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate

```
mysql> SELECT C.CustomerName,
    ->        O.ProductName,
    ->        O.Amount
    -> FROM Customer C
    -> JOIN Orders O
    -> ON C.CustomerID = O.CustomerID;
+--------------+-------------+--------+
| CustomerName | ProductName | Amount |
+--------------+-------------+--------+
| Arun         | Laptop      |  55000 |
| Priya        | Mobile      |  25000 |
| Rahul        | Printer     |  12000 |
| Divya        | Monitor     |  18000 |
+--------------+-------------+--------+
4 rows in set (0.00 sec)

mysql> SELECT
    -> CustomerName,
    -> (
    -> SELECT ProductName
    -> FROM Orders
    -> WHERE Orders.CustomerID = Customer.CustomerID
    -> ) AS ProductName
    -> FROM Customer;
+--------------+-------------+
| CustomerName | ProductName |
+--------------+-------------+
| Arun         | Laptop      |
| Priya        | Mobile      |
| Rahul        | Printer     |
| Divya        | Monitor     |
+--------------+-------------+
4 rows in set (0.05 sec)
```

```
mysql> SELECT * FROM Customer;
+------------+--------------+------------+
| CustomerID | CustomerName | City       |
+------------+--------------+------------+
|        101 | Arun         | Chennai    |
|        102 | Priya        | Madurai    |
|        103 | Rahul        | Coimbatore |
|        104 | Divya        | Salem      |
+------------+--------------+------------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Orders;
+---------+------------+-------------+--------+
| OrderID | CustomerID | ProductName | Amount |
+---------+------------+-------------+--------+
|       1 |        101 | Laptop      |  55000 |
|       2 |        102 | Mobile      |  25000 |
|       3 |        103 | Printer     |  12000 |
|       4 |        104 | Monitor     |  18000 |
+---------+------------+-------------+--------+
4 rows in set (0.00 sec)

mysql> SELECT C.CustomerName,
    ->        O.ProductName,
    ->        O.Amount
    -> FROM Customer C
    -> JOIN Orders O
    -> ON C.CustomerID = O.CustomerID;
+--------------+-------------+--------+
| CustomerName | ProductName | Amount |
+--------------+-------------+--------+
| Arun         | Laptop      |  55000 |
| Priya        | Mobile      |  25000 |
| Rahul        | Printer     |  12000 |
| Divya        | Monitor     |  18000 |
+--------------+-------------+--------+
4 rows in set (0.00 sec)
```

```
mysql> CREATE DATABASE ShopDB;
Query OK, 1 row affected (0.09 sec)

mysql> USE ShopDB;
Database changed
mysql> CREATE TABLE Customer
    -> (
    ->     CustomerID INT PRIMARY KEY,
    ->     CustomerName VARCHAR(50),
    ->     City VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Orders
    -> (
    ->     OrderID INT PRIMARY KEY,
    ->     CustomerID INT,
    ->     ProductName VARCHAR(50),
    ->     Amount INT,
    ->     FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Customer VALUES
    -> (101,'Arun','Chennai'),
    -> (102,'Priya','Madurai'),
    -> (103,'Rahul','Coimbatore'),
    -> (104,'Divya','Salem');
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Orders VALUES
    -> (1,101,'Laptop',55000),
    -> (2,102,'Mobile',25000),
    -> (3,103,'Printer',12000),
    -> (4,104,'Monitor',18000);
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

6.Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create

```
mysql> CREATE DATABASE LibraryDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE LibraryDB;
Database changed
mysql> CREATE TABLE Books
    -> (
    ->     BookID INT PRIMARY KEY,
    ->     BookName VARCHAR(50),
    ->     Author VARCHAR(50),
    ->     Price INT,
    ->     Quantity INT
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO Books VALUES
    -> (101,'Database Systems','Navathe',550,10),
    -> (102,'Java Programming','Herbert Schildt',650,15),
    -> (103,'Python Basics','Mark Lutz',500,20),
    -> (104,'Operating Systems','Galvin',700,8);
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Books;
+--------+------------------+-----------------+-------+----------+
| BookID | BookName         | Author          | Price | Quantity |
+--------+------------------+-----------------+-------+----------+
|    101 | Database Systems | Navathe         |   550 |       10 |
|    102 | Java Programming | Herbert Schildt |   650 |       15 |
|    103 | Python Basics    | Mark Lutz       |   500 |       20 |
|    104 | Operating Systems| Galvin          |   700 |        8 |
+--------+------------------+-----------------+-------+----------+
4 rows in set (0.00 sec)

mysql> CREATE VIEW Book_View AS
    -> SELECT
    -> BookID,
    -> BookName,
    -> Author
    -> FROM Books;
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> SELECT * FROM Book_View;
+--------+------------------+-----------------+
| BookID | BookName         | Author          |
+--------+------------------+-----------------+
|    101 | Database Systems | Navathe         |
|    102 | Java Programming | Herbert Schildt |
|    103 | Python Basics    | Mark Lutz       |
|    104 | Operating Systems| Galvin          |
+--------+------------------+-----------------+
4 rows in set (0.05 sec)

mysql> UPDATE Books
    -> SET Quantity = 25
    -> WHERE BookID = 103;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Books;
+--------+------------------+-----------------+-------+----------+
| BookID | BookName         | Author          | Price | Quantity |
+--------+------------------+-----------------+-------+----------+
|    101 | Database Systems | Navathe         |   550 |       10 |
|    102 | Java Programming | Herbert Schildt |   650 |       15 |
|    103 | Python Basics    | Mark Lutz       |   500 |       25 |
|    104 | Operating Systems| Galvin          |   700 |        8 |
+--------+------------------+-----------------+-------+----------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Book_View;
+--------+------------------+-----------------+
| BookID | BookName         | Author          |
+--------+------------------+-----------------+
|    101 | Database Systems | Navathe         |
|    102 | Java Programming | Herbert Schildt |
|    103 | Python Basics    | Mark Lutz       |
|    104 | Operating Systems| Galvin          |
+--------+------------------+-----------------+
4 rows in set (0.00 sec)
```

7.Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze

```
mysql> CREATE DATABASE HospitalDB;
Query OK, 1 row affected (0.07 sec)

mysql> USE HospitalDB;
Database changed
mysql> CREATE TABLE Patient
    -> (
    ->     PatientID INT PRIMARY KEY,
    ->     PatientName VARCHAR(50),
    ->     Age INT
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Treatment
    -> (
    ->     TreatmentID INT PRIMARY KEY,
    ->     PatientID INT,
    ->     DoctorName VARCHAR(50),
    ->     Disease VARCHAR(50),
    ->     FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO Patient VALUES
    -> (101,'Ravi',25),
    -> (102,'Anita',30),
    -> (103,'Kumar',40),
    -> (104,'Divya',28);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Treatment VALUES
    -> (1,101,'Dr. Raj','Fever'),
    -> (2,102,'Dr. Meena','Diabetes'),
    -> (3,103,'Dr. Raj','Cold'),
    -> (4,104,'Dr. Arun','Fever');
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Patient;
+-----------+-------------+------+
| PatientID | PatientName | Age  |
+-----------+-------------+------+
|       101 | Ravi        |   25 |
|       102 | Anita       |   30 |
|       103 | Kumar       |   40 |
|       104 | Divya       |   28 |
+-----------+-------------+------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Treatment;
+-------------+-----------+------------+----------+
| TreatmentID | PatientID | DoctorName | Disease  |
+-------------+-----------+------------+----------+
|           1 |       101 | Dr. Raj    | Fever    |
|           2 |       102 | Dr. Meena  | Diabetes |
|           3 |       103 | Dr. Raj    | Cold     |
|           4 |       104 | Dr. Arun   | Fever    |
+-------------+-----------+------------+----------+
4 rows in set (0.00 sec)
```

```
mysql> SELECT P.PatientName
    -> FROM Patient P
    -> JOIN Treatment T
    -> ON P.PatientID = T.PatientID
    -> WHERE T.Disease = 'Fever';
+-------------+
| PatientName |
+-------------+
| Ravi        |
| Divya       |
+-------------+
2 rows in set (0.00 sec)
```

8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate

```
mysql> SELECT * FROM Movie;
+---------+---------------+----------+
| MovieID | MovieName     | Language |
+---------+---------------+----------+
|     101 | Leo           | Tamil    |
|     102 | Jailer        | Tamil    |
|     103 | Kalki 2898 AD | Telugu   |
|     104 | Pushpa 2      | Telugu   |
+---------+---------------+----------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Booking;
+-----------+---------+--------------+-------------+
| BookingID | MovieID | CustomerName | TicketCount |
+-----------+---------+--------------+-------------+
|         1 |     101 | Arun         |           2 |
|         2 |     102 | Priya        |           4 |
|         3 |     103 | Rahul        |           3 |
|         4 |     104 | Divya        |           5 |
+-----------+---------+--------------+-------------+
4 rows in set (0.00 sec)

mysql> SELECT CustomerName,
    -> (
    -> SELECT MovieName
    -> FROM Movie
    -> WHERE Movie.MovieID = Booking.MovieID
    -> ) AS Movie
    -> FROM Booking;
+--------------+---------------+
| CustomerName | Movie         |
+--------------+---------------+
| Arun         | Leo           |
| Priya        | Jailer        |
| Rahul        | Kalki 2898 AD |
| Divya        | Pushpa 2      |
+--------------+---------------+
4 rows in set (0.00 sec)
```

```
mysql> SELECT B.CustomerName,
    ->        M.MovieName,
    ->        B.TicketCount
    -> FROM Booking B
    -> JOIN Movie M
    -> ON B.MovieID = M.MovieID;
+--------------+---------------+-------------+
| CustomerName | MovieName     | TicketCount |
+--------------+---------------+-------------+
| Arun         | Leo           |           2 |
| Priya        | Jailer        |           4 |
| Rahul        | Kalki 2898 AD |           3 |
| Divya        | Pushpa 2      |           5 |
+--------------+---------------+-------------+
4 rows in set (0.00 sec)

mysql> CREATE INDEX idx_movieid
    -> ON Booking(MovieID);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE VIEW Booking_Details AS
    -> SELECT B.CustomerName,
    ->        M.MovieName,
    ->        M.Language,
    ->        B.TicketCount
    -> FROM Booking B
    -> JOIN Movie M
    -> ON B.MovieID = M.MovieID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Booking_Details;
+--------------+---------------+----------+-------------+
| CustomerName | MovieName     | Language | TicketCount |
+--------------+---------------+----------+-------------+
| Arun         | Leo           | Tamil    |           2 |
| Priya        | Jailer        | Tamil    |           4 |
| Rahul        | Kalki 2898 AD | Telugu   |           3 |
| Divya        | Pushpa 2      | Telugu   |           5 |
+--------------+---------------+----------+-------------+
4 rows in set (0.00 sec)
```

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create

```
mysql> CREATE DATABASE SalesDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE SalesDB;
Database changed
mysql> CREATE TABLE Sales
    -> (
    ->     SaleID INT PRIMARY KEY,
    ->     SalesPerson VARCHAR(50),
    ->     Product VARCHAR(50),
    ->     Quantity INT,
    ->     Amount INT
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO Sales VALUES
    -> (101,'Arun','Laptop',2,100000),
    -> (102,'Priya','Mobile',5,125000),
    -> (103,'Rahul','Laptop',3,150000),
    -> (104,'Divya','Printer',4,40000),
    -> (105,'Arun','Mobile',2,50000),
    -> (106,'Priya','Laptop',1,50000);
Query OK, 6 rows affected (0.05 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Sales;
+--------+-------------+---------+----------+--------+
| SaleID | SalesPerson | Product | Quantity | Amount |
+--------+-------------+---------+----------+--------+
|    101 | Arun        | Laptop  |        2 | 100000 |
|    102 | Priya       | Mobile  |        5 | 125000 |
|    103 | Rahul       | Laptop  |        3 | 150000 |
|    104 | Divya       | Printer |        4 |  40000 |
|    105 | Arun        | Mobile  |        2 |  50000 |
|    106 | Priya       | Laptop  |        1 |  50000 |
+--------+-------------+---------+----------+--------+
6 rows in set (0.00 sec)
```

```
mysql> SELECT SalesPerson,
    ->        SUM(Amount) AS TotalSales
    -> FROM Sales
    -> GROUP BY SalesPerson
    -> HAVING SUM(Amount) >
    -> (
    ->     SELECT AVG(TotalSales)
    ->     FROM
    ->     (
    ->         SELECT SUM(Amount) AS TotalSales
    ->         FROM Sales
    ->         GROUP BY SalesPerson
    ->     ) AS SalesSummary
    -> );
+-------------+------------+
| SalesPerson | TotalSales |
+-------------+------------+
| Arun        |     150000 |
| Priya       |     175000 |
| Rahul       |     150000 |
+-------------+------------+
3 rows in set (0.00 sec)

mysql>
```

10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate

```
mysql> CREATE DATABASE HotelDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE HotelDB;
Database changed
mysql> CREATE TABLE Hotel
    -> (
    ->     HotelID INT PRIMARY KEY,
    ->     HotelName VARCHAR(50),
    ->     City VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Booking
    -> (
    ->     BookingID INT PRIMARY KEY,
    ->     HotelID INT,
    ->     CustomerName VARCHAR(50),
    ->     Amount INT,
    ->     FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO Hotel VALUES
    -> (101,'Grand Palace','Chennai'),
    -> (102,'Sea View','Madurai'),
    -> (103,'Royal Inn','Coimbatore'),
    -> (104,'Green Park','Salem');
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Booking VALUES
    -> (1,101,'Arun',3000),
    -> (2,102,'Priya',2500),
    -> (3,103,'Rahul',4000),
    -> (4,104,'Divya',3500);
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Hotel;
+---------+--------------+-------------+
| HotelID | HotelName    | City        |
+---------+--------------+-------------+
|     101 | Grand Palace | Chennai     |
|     102 | Sea View     | Madurai     |
|     103 | Royal Inn    | Coimbatore  |
|     104 | Green Park   | Salem       |
+---------+--------------+-------------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Booking;
+-----------+---------+--------------+---------+
| BookingID | HotelID | CustomerName | Amount  |
+-----------+---------+--------------+---------+
|         1 |     101 | Arun         |   3000  |
|         2 |     102 | Priya        |   2500  |
|         3 |     103 | Rahul        |   4000  |
|         4 |     104 | Divya        |   3500  |
+-----------+---------+--------------+---------+
4 rows in set (0.00 sec)

mysql> SELECT H.HotelName,
    ->        B.CustomerName,
    ->        B.Amount
    -> FROM Hotel H
    -> JOIN Booking B
    -> ON H.HotelID = B.HotelID;
+--------------+--------------+---------+
| HotelName    | CustomerName | Amount  |
+--------------+--------------+---------+
| Grand Palace | Arun         |   3000  |
| Sea View     | Priya        |   2500  |
| Royal Inn    | Rahul        |   4000  |
| Green Park   | Divya        |   3500  |
+--------------+--------------+---------+
4 rows in set (0.00 sec)
```

```
mysql> SELECT CustomerName,
    -> (
    -> SELECT HotelName
    -> FROM Hotel
    -> WHERE Hotel.HotelID = Booking.HotelID
    -> ) AS HotelName
    -> FROM Booking;
+--------------+--------------+
| CustomerName | HotelName    |
+--------------+--------------+
| Arun         | Grand Palace |
| Priya        | Sea View     |
| Rahul        | Royal Inn    |
| Divya        | Green Park   |
+--------------+--------------+
4 rows in set (0.00 sec)

mysql> CREATE VIEW Booking_Details AS
    -> SELECT H.HotelName,
    ->        B.CustomerName,
    ->        B.Amount
    -> FROM Hotel H
    -> JOIN Booking B
    -> ON H.HotelID = B.HotelID;
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT * FROM Booking_Details;
+--------------+--------------+---------+
| HotelName    | CustomerName | Amount  |
+--------------+--------------+---------+
| Grand Palace | Arun         |   3000  |
| Sea View     | Priya        |   2500  |
| Royal Inn    | Rahul        |   4000  |
| Green Park   | Divya        |   3500  |
+--------------+--------------+---------+
4 rows in set (0.00 sec)
```